

B U P T

密码工程

第8讲、国密算法及其实现 SM3、SM4和祖冲之算法

授课教师：王晨宇

时间：2026年4月23日



目录

contents

B U P T



4.1 SM3算法

4.2 SM4算法

4.3 ZUC算法

4.4 国密算法库介绍

4.1

SM3算法

▶ SM3算法

B U P T



✓ 4.1.1 SM3简介

✓ 4.1.2 SM3结构

✓ 4.1.3 填充算法

✓ 4.1.4 消息扩展

✓ 4.1.5 压缩函数

✓ 4.1.6 与其他哈希函数对比



SM3算法

4.1.1 SM3简介

B U P T

- SM3是中华人民共和国采用的一种密码散列函数标准，是由中国国家密码管理局与2010年公布的中国商用密码杂凑算法标准。该算法于2012年发布为密码行业标准(GM/T 0004-2012)，2016年发布为国家密码杂凑算法标准(GB/T 32905-2016)。
- 在商用密码体系中，SM3主要用于数字签名及验证、消息认证码生成及验证、随机数生成等，其安全性与效率和SHA-256相当。

ICS 35.040
L 80
备案号:36831—2012



中华人民共和国密码行业标准

GM/T 0004—2012

SM3 密码杂凑算法

SM3 cryptographic hash algorithm

2012-03-21 发布

2012-03-21 实施

国家密码管理局 发布

ICS 35.040
L 80



中华人民共和国国家标准

GB/T 32905—2016

信息安全技术 SM3 密码杂凑算法

Information security techniques—SM3 cryptographic hash algorithm

2016-08-29 发布

2017-03-01 实施

中华人民共和国国家质量监督检验检疫总局
中国国家标准化管理委员会 发布

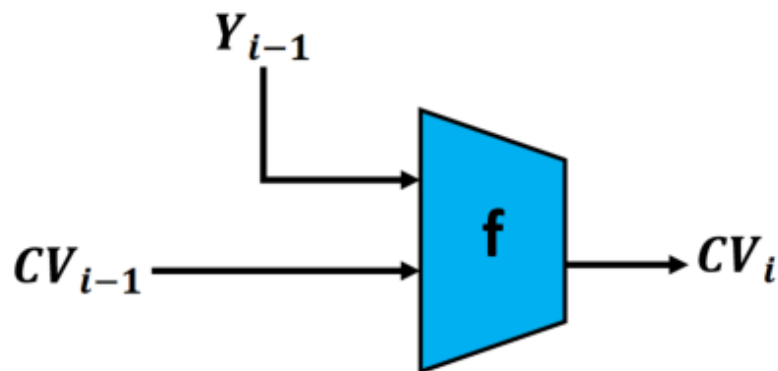


- SM3和MD5的迭代过程类似，采用Merkle-Damgard结构。
- 所需的安全散列长度越来越长，我们可以使用有限定义域上的散列函数(俗称压缩函数)通过迭代方式拓展为具有无限定义域的散列函数，而最具有代表性的就是Merkle-Damgard结构。
- 如果压缩函数是抗碰撞的，那经过Merkle-Damgard结构处理后的散列函数也是抗碰撞的。Merkle-Damgard结构如下：

$$CV_0 = IV$$

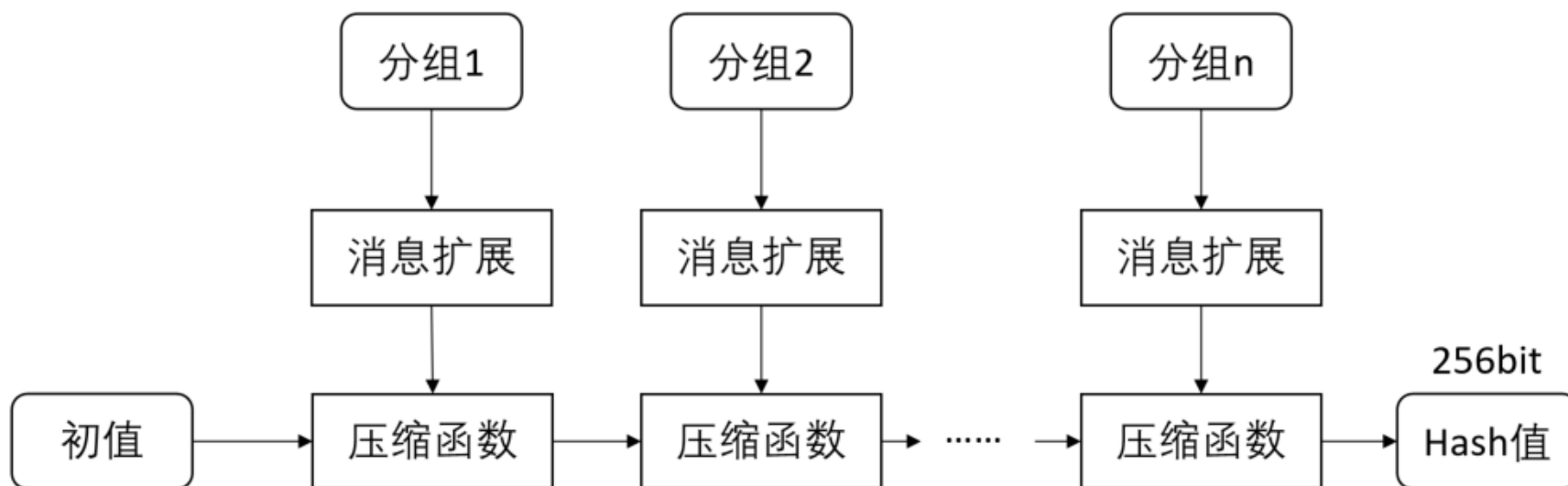
$$CV_i = f(CV_{i-1}, Y_{i-1})$$

$$H(M) = CV_i$$



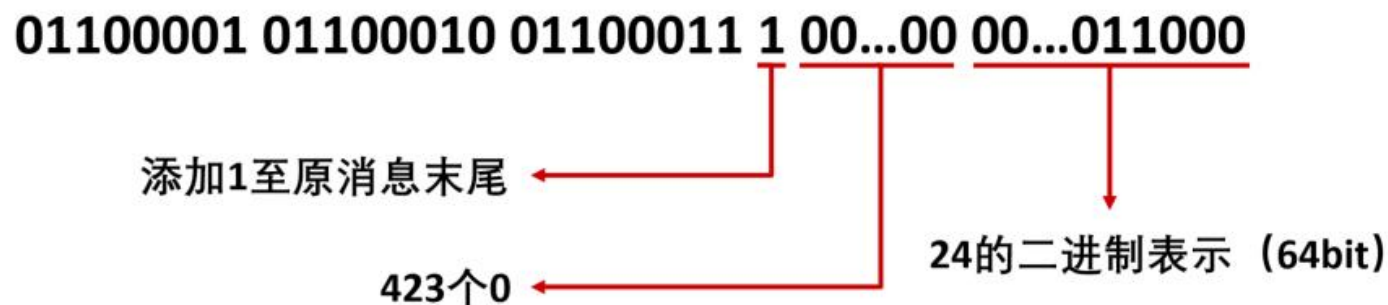


- SM3算法的输入为长度 $L(L < 2^{64})$ 的比特消息 m ，经过填充、分组、迭代压缩后生成哈希值，分组为512bit，哈希值长度为256bit。





- 将比特“1”添加至消息末尾，再添加k个0($l + 1 + k \equiv 448 \pmod{512}$), 最后添加一个64位bit串表示消息的长度L。
- 例如：对消息01100001 01100010 01100011，其长度为24，填充后的比特串为：





□ 将 B_i 划分为16个字表示(16个32比特):

$$B_i \rightarrow W_0 \dots W_{15}$$

□ 循环生成 W_i , $\ll$ 表示32bit循环左移:

for $i = 16$ to 67

$$W_i \leftarrow P_1(W_{i-16} \oplus W_{i-9} \oplus (W_{i-3} \ll 15)) \oplus (W_{i-13} \ll 7) \oplus W_{i-6}$$

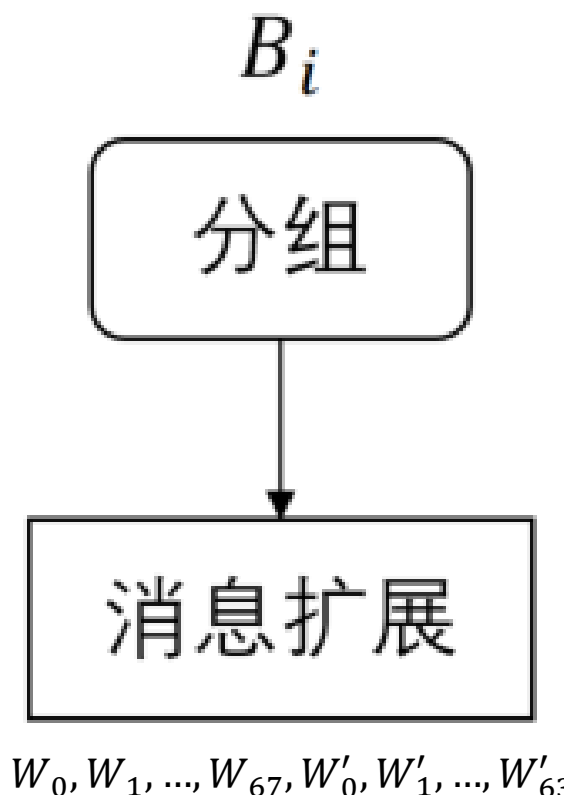
□ 循环生成 W'_i :

for $i = 0$ to 63

$$W'_i = W_i \oplus W_{i+4}$$

□ 置换函数 P_1 :

$$P_1(X) = X \oplus (X \ll 15) \oplus (X \ll 23)$$





□ A, B, C, D, E, F, G, H为字寄存器(大端格式存储), SS1, SS2, TT1, TT2为中间变量。使用压缩函数时有通式:

$$V^{i+1} = CF(V^i, B^i), 0 \leq i \leq n - 1$$

□ 其具体计算过程如下:





□ 布尔函数 FF_i 与 GG_i :

$$FF_i(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z) & 16 \leq j \leq 63 \end{cases}$$

$$GG_i(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (\sim X \wedge Z) & 16 \leq j \leq 63 \end{cases}$$

□ 置换函数 P_0 :

$$P_0(X) = X \oplus (X \ll 9) \oplus (X \ll 17)$$

□ 压缩函数中用到的常量 T_j :

$$T_j = \begin{cases} 79cc4515 & 0 \leq j \leq 15 \\ 7a879d8a & 16 \leq j \leq 63 \end{cases}$$



□ 衡量加密算法的安全性一个关键指标是密钥强度，密钥强度为k意味着破解密钥的计算复杂度为2的k次方。强度为128就是说目前最好的攻击算法需要计算2的128次方才能攻破。

算法	输入位数	输出位数	迭代次数	分组长度	强度
MD5	无限	128	64	512	
SHA-1	264	160	80	512	60
SHA-224	264	224	64	512	112
SHA-256	264	256	64	512	128
SHA-384	2 128	384	80	1 024	192
SHA-512	2 128	512	80	1 024	256
SM3	264	256	64	512	128

□ SM3算法的安全性与SHA256算法的安全性相当。



课堂思考

B U P T

1.SM3可以应用在哪些领域?

4.2

SM4算法

▶ SM4算法



✓ 4.2.1 SM4简介

✓ 4.2.2 SM4结构

✓ 4.2.3 SM4加/解密过程

✓ 4.2.4 密钥扩展

✓ 4.2.5 密钥强度对比

✓ 4.2.6 密钥强度建议值



SM4算法

4.2.1 SM4简介

B U P T

- SM4.0(原名SMS4.0)由国家密码管理局于2012年3月21日发布, 相关标准为“GM/T 0002-2012 《SM4 分组密码算法》”。
- 在商用密码体系中, SM4主要用于数据加密, 分组长度与密钥长度均为128比特, 加密算法与密钥扩展算法都采用32轮**非线性迭代结构**。SM4密码算法以字节(8位)和字(32位)作为单位进行数据处理。

ICS 35.040
L 80
备案号:36825—2012



中华人民共和国密码行业标准

GM/T 0002—2012

SM4 分组密码算法

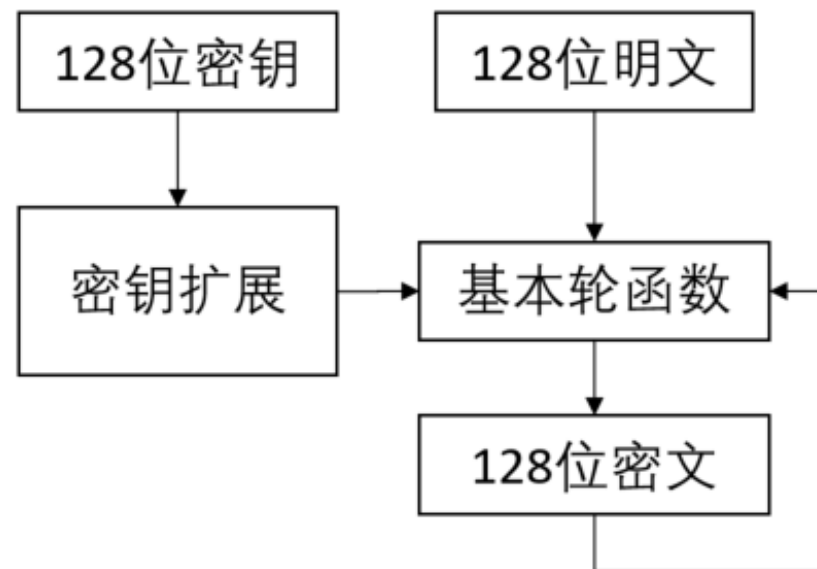
SM4 block cipher algorithm

2012-03-21 发布

2012-03-21 实施



- SM4密码算法经过我国专业密码机构的充分分析测试，可以抵抗**差分攻击**、**线性攻击**等现有攻击，因此是安全的。其中S盒在非线性度、自相关性、代数免疫性等方面有相当高的水平。
- 对合运算指的是一个函数，其自身的逆函数也是它自己。换句话说，对于定义域中的所有 x ，有 $f(f(x)) = x$
- SM4密码算法是对合运算，因此解密算法与加密算法的结构相同，只是轮密钥的使用顺序相反。SM4结构如右图所示。





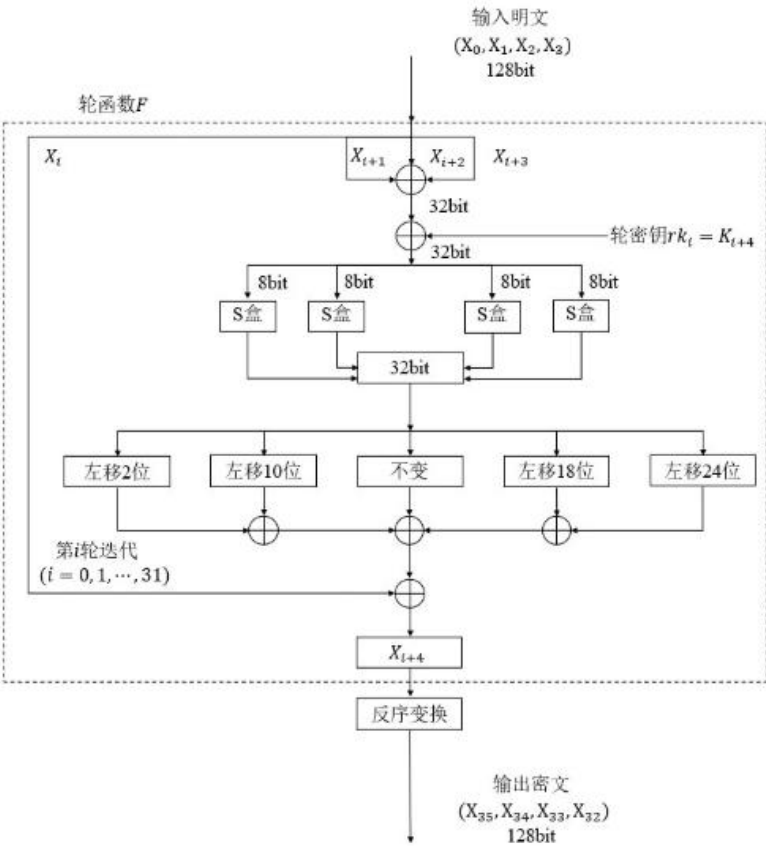
□在SM4加解密过程和密钥扩展过程中用到了S盒，具体如下：

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
A	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
B	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
C	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
D	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
E	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
F	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

注：输入‘EF’，则经S盒后的值为表中第E行和第F列的值，Sbox(EF) = 84。



SM4加/解密流程如下图所示：





- 输入的明文长度位128bit, 先将其拆分为4个32bit的数据:

$$X_i, X_{i+1}, X_{i+2}, X_{i+3}$$

- 其中, X_i 先不做处理, 将 $X_{i+1}, X_{i+2}, X_{i+3}$ 与轮密钥 rk_i 进行异或操作, 并将结果作为盒运算的输入(32bit), 即

$$SboxInput = X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i$$

- 将SboxInput拆分为4个8bit数据, 分别进行盒运算, 并将盒运算后的4个8bit重新合成一个32bit数据SboxOutput。
- 对SboxOutput分别进行五种位移操作(左移2位、左移10位、不移位、左移18位、左移24位), 将五种结果与 X_i 进行异或操作, 其结果作为一轮加密的输出 X_{i+4} 。

$$\begin{aligned} output1 = & (SboxOutput \ll 2) \oplus (SboxOutput \ll \\ & \ll 10) \oplus (SboxOutput) \oplus (SboxOutput \ll 18) \oplus (SboxOutput \ll 24) \oplus X_i \end{aligned}$$

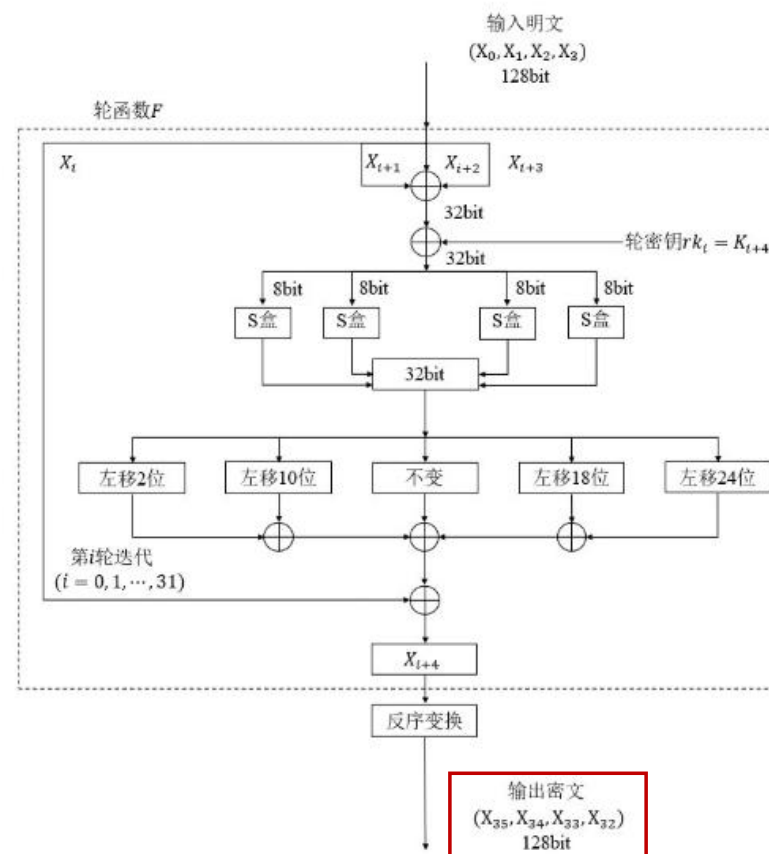


SM4算法

4.2.3 SM4加解密过程(3)

B U P T

- 得到 X_{i+4} ，至此完成了一轮加密，在实际加密过程中，该过程要进行32轮，每一轮使用不同的轮密钥 rk_i 参与加密，每一轮的 rk_i 由密钥扩展部分生成。
- 经过32轮加密后，将生成的最后4个32bit数据(在图中为 $X_{35}, X_{34}, X_{33}, X_{32}$ ，最后要进行反序变换，所以是按 $X_{35} \sim X_{32}$ 的顺序)合并成一个128bit的数据output作为加密后的结果。





- 对于解密，一个简单的方法是将轮密钥 rk_i 逆序后再执行一次32轮的加密运算，即将密文投入加密函数，并且第0轮使用 rk_{31} 作为轮密钥，第 i 轮使用 rk_{31-i} 作为轮密钥，最后获得的结果便是加密前的密文。过程如下所示：

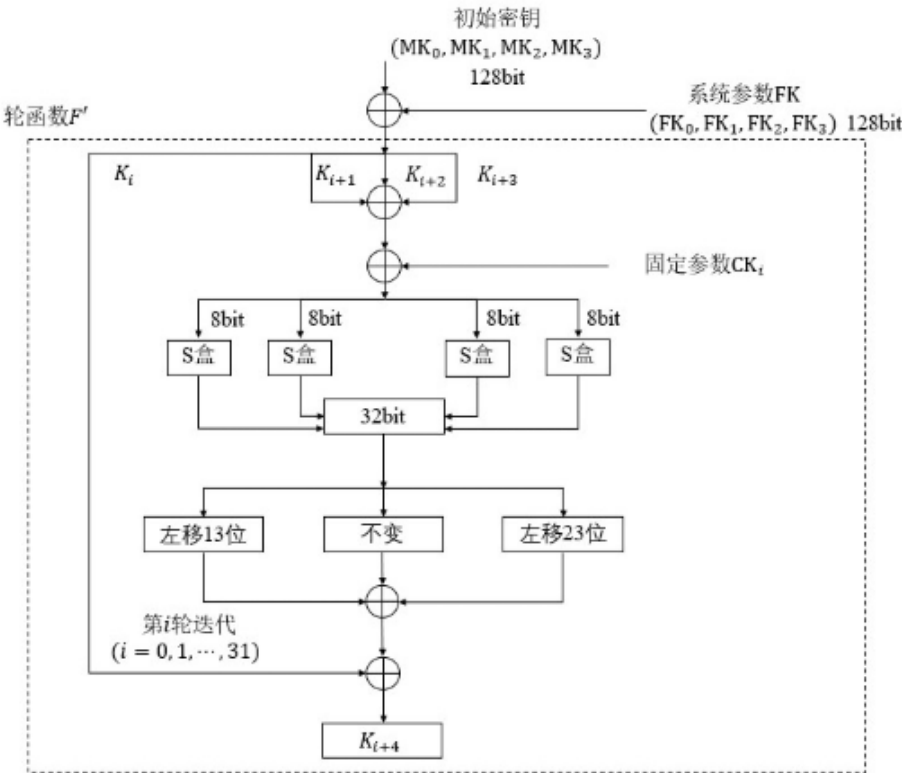
接下来看解密，记加密的明文为 x_0, x_1, x_2, x_3 ，将加密算法最后输出结果 $x_{35}, x_{34}, x_{33}, x_{32}$ 记为 x'_0, x'_1, x'_2, x'_3 ，并让 $rk'_i = rk_{31-i}$ (也就是轮密钥逆序)

将 x'_0, x'_1, x'_2, x'_3 和 rk'_i 投入加密算法中，观察会发生什么结果

- 对于第 0 轮生成的 $x'_4 = x'_0 \oplus T(x'_1 \oplus x'_2 \oplus x'_3 \oplus rk'_0)$ ，我们知道 $x'_0 = x_{35}, x'_1 = x_{34}, x'_2 = x_{33}, x'_3 = x_{32}$ ，故 $x'_4 = x_{35} \oplus T(x_{34} \oplus x_{33} \oplus x_{32} \oplus rk_{31}) = x_{31} \oplus T(\dots) \oplus T(\dots) = x_{31}$
- 类似的，我们可以得出 $x'_i = x_{35-i}$ ，最后一轮函数结束后的结果为 x_3, x_2, x_1, x_0 ，经过逆序后便是 x_0, x_1, x_2, x_3 ，刚好是加密前的明文，完成了解密操作



□ 密钥扩展的流程如下图所示：





- 密钥扩展部分和加解密部分类似，都要经过拆分、盒变换、移位、异或等操作。
- 输入初始密钥长度为128bit，先将其拆分为4个32bit的数据：

$$MK_i, MK_{i+1}, MK_{i+2}, MK_{i+3}$$

- 对每一个32bit的数据 MK_i ，将其与128bit的系统参数FK的 FK_i 进行异或，得到 $K_i, K_{i+1}, K_{i+2}, K_{i+3}$ 。

$$SboxInput = K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i$$

- 将SboxInput拆分为4个8bit数据，分别进行盒运算，并将盒运算后的4个8bit重新合并成一个32bit数据SboxOutput。
- 对SboxOutput分别进行3种移位操作(左移13位，不移位，左移32位，将三种结果与 K_i 进行异或操作，其结果作为一轮加密的输出 K_{i+4})。

$$output1 = (SboxOutput \ll 13) \oplus (SboxOutput) \oplus (SboxOutput \ll 23) \oplus K_i$$



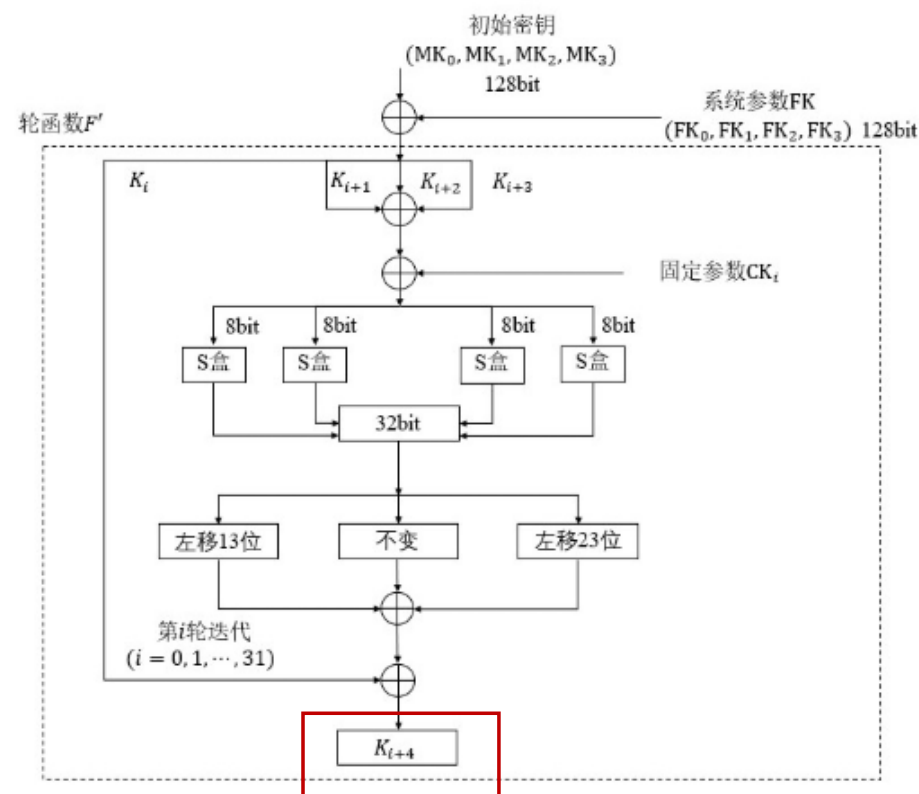
SM4算法

4.2.4 密钥扩展(3)

B U P T

□得到 $rk_i = K_{i+4}$ 。至此完成了一轮密钥生成，在实际加密过程中，该过程要执行32轮，每一轮使用不同的固定参数 CK_i 参与加密。

□执行完32轮后，便可获得32个用于加解密的 rk_i 。





□ SM4与其他对称密钥算法对比如下，可以看出SM4的安全性与AES128相当。

算法	密钥长度	分组长度	循环次数	强度
DES	56	64	16	
3DES-2	112	64	16*3	80 ~ 112
3DES-3	168	64	16*3	112
AES-128	128	128	10	128
AES-256	256	128	14	256
SM4	128	128	32	128



■ ECRYPT II Yearly Report on Algorithms and Keysizes （2011-2012） 密钥强度的建议值，根据国际建议值, 未来达到有效的安全保护, 金融级应用的加密算法的密钥强度应该达到112bit, 长期的保护要求要达到128bit甚至256bit。

安全性	密钥强度
抵御个人实时攻击	32
小型组织的攻击保护	64
中等规模组织的攻击保护	72
专业机构的简易攻击保护	80
老的标准	96
中期保护要求	112
长期保护要求	128
可预见未来	256



■ ECRYPT-CSA (Coordination & Support Action)的目标是加强欧洲在密码学领域的卓越性。 本报告旨在成为该领域的参考资料，重点关注收集、存储和处理数据的商业在线服务。

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2022-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2022-2068)</i>	256	15360	512	15360	512	512

[1] <https://www.keylength.com/en/3/>



针对未来密钥强度的要求, 美国国家标准与技术研究院 (NIST)、法国信息系统安全局 (ANSSI) 和德国联邦信息安全办公室 (BSI) 给出了未来几十年的密钥强度建议值。

组织	时间段	密钥强度	
		对称算法	非对称算法
ANSSI (France)	2020以前	100	100
	2021年-2030年	128	100
	2031年以后	128	128
NIST (US)	2030年以前	112	
	2030年以后	128	
BSI (Germany)	2015年以前	112	100 (RSA) 112 (ECC)
	2016年-2019年	128	100 (RSA) 112 (ECC)
	2020以后	128	128



SM4算法

4.2.6 密钥强度建议值(3)

B U P T

美国国家安全局 (National Security Agency, NSA) 公布了其密码标准要求，对于SECRET级别的密钥强度要求128bit, 对于TOP SECRET级别的密钥强度要求达到192bit。

算法	功能	密钥长度要求	
		SECRET	TOP SECRET
Advanced Encryption Standard (AES)	Encryption	128 bit	256 bit
Elliptic Curve Diffie-Hellman (ECDH)	Key Exchange	Curve P-256	Curve P-384
Elliptic Curve Digital Signature Algorithm (ECDSA)	Digital Signature	Curve P-256	Curve P-384
Secure Hash Algorithm (SHA)	Hashing	SHA-256	SHA-384



课堂思考

B U P T

2.SM4的安全性体现在哪些方面?

4.3

ZUC算法

▶ ZUC算法



✓ 4.3.1 ZUC简介

✓ 4.3.2 ZUC结构

✓ 4.3.3 比特重组BR

✓ 4.3.4 非线性函数

✓ 4.3.5 ZUC算法过程

✓ 4.3.6 ZUC的安全性



- 祖冲之算法集是由我国学者自主设计的加密和完整性算法，包括祖冲之算法（ZUC）、加密算法128-EEA3和完整性算法128-EIA3。ZUC 算法属于同步序列密码（可替换RC4算法），也是中国第一个成为国际密码标准的密码算法。
- 2009年5月ZUC算法获得3GPP安全算法组SA立项，正式申请参加3GPP LTE 第三套机密性和完整性算法标准的竞选工作。2011年9月19-21日，在日本福冈召开的第53次第三代合作伙伴（3GPP）系统架构组（SA）会议上，祖冲之密码算法（ZUC）被批准成为新一代宽带无线移动通信系统（LTE）国际标准，即4G的国际标准。
- 其标准化的成功，是中国在商用密码算法领域取得的一次重大突破，体现了中国商用密码应用的开放性和商用密码设计的高能力，增大了中国在国际通信安全应用领域的影响力，无论是对中国在国际商用密码标准化方面的工作还是商用密码的密码设计来说都有深远的影响。

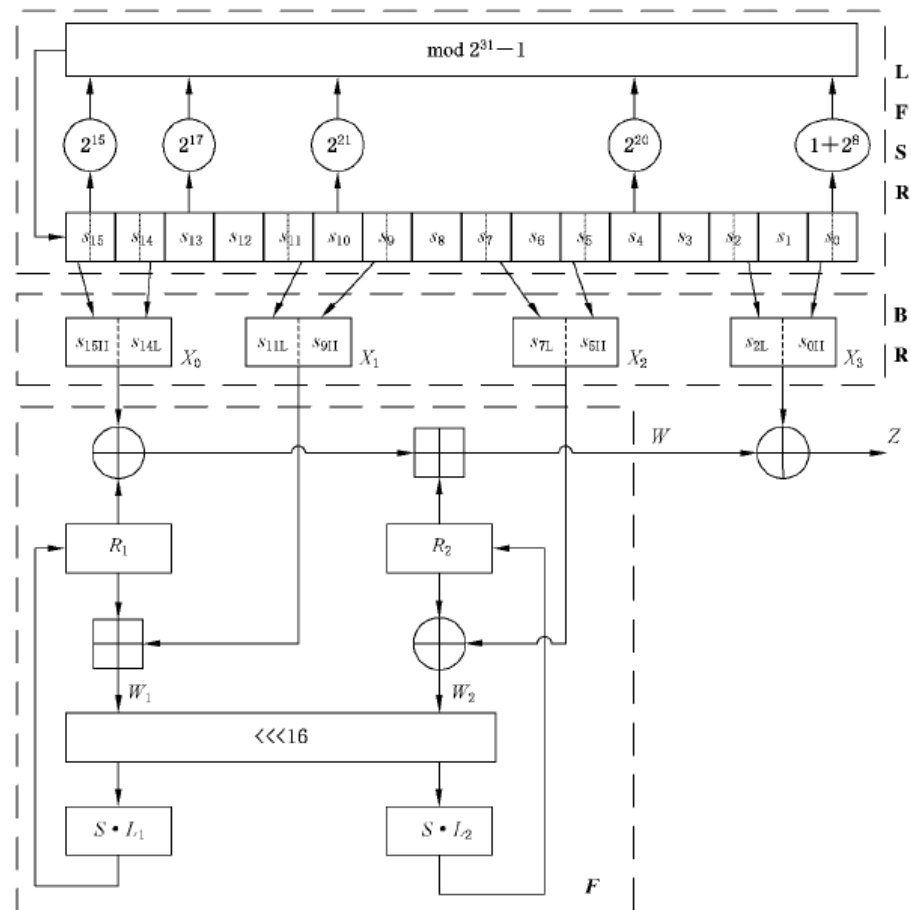


ZUC算法

4.3.2 ZUC算法结构

B U P T

- 面向字的流密码：它采用128位的初始密钥作为输入、一个128位的初始向量IV，并输出关于字的密钥流（从而每32位被称为一个密钥字）。密钥流可用于对信息进行加密/解密。
- LFSR 包括 16 个 31 比特寄存器单元变量 $S_0, S_1, \dots, S_{15}$ 。
- ZUC的执行分为两个阶段：初始化阶段和工作阶段。





- 在第一阶段，密钥和初始向量进行初始化，即不产生输出。
- 第二个阶段是工作阶段，在这个阶段，每一个时钟脉冲产生一个32bit的密钥输出。
- ZUC算法在逻辑上分为上中下三层：
 - 上层是16级线性反馈移位寄存器LFSR；
 - 中层是比特重组BR；
 - 下层是非线性函数F。



- 在初始化模式下，在初始化模式下，LFSR接收一个31比特字 u 。 u 是由非线性函数 F 的32比特输出 v 通过舍弃最低位比特得到，目的是使线性反馈移位寄存器的状态随机化。

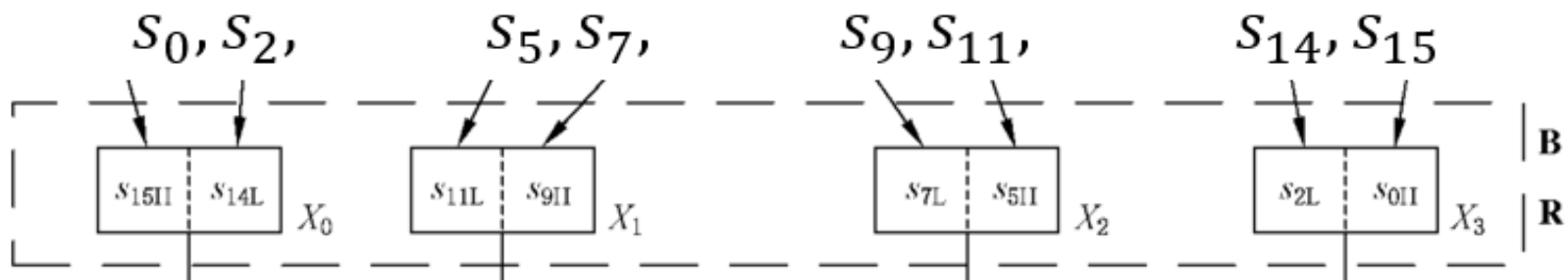
```
LFSRWithInitialisationMode( $u$ )  
{  
    (1)  $v = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0 \bmod (2^{31} - 1)$ ;  
    (2)  $s_{16} = (v + u) \bmod (2^{31} - 1)$ ;  
    (3) 如果  $s_{16} = 0$ , 则置  $s_{16} = 2^{31} - 1$ ;  
    (4)  $(s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15})$ 。  
}
```

- 在工作模式下，LFSR不接受任何输入。

```
LFSRWithWorkMode()  
{  
    (1)  $s_{16} = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0 \bmod (2^{31} - 1)$ ;  
    (2) 如果  $s_{16} = 0$ , 则置  $s_{16} = 2^{31} - 1$ ;  
    (3)  $(s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15})$ 。  
}
```



□ 输入为LFSR寄存器单元变量：



□ 具体如下所示：

BitReconstruction()

{

$$(1) X_0 = s_{15H} \parallel s_{14L};$$

$$(2) X_1 = s_{11L} \parallel s_{9H};$$

$$(3) X_2 = s_{7L} \parallel s_{5H};$$

$$(4) X_3 = s_{2L} \parallel s_{0H}.$$

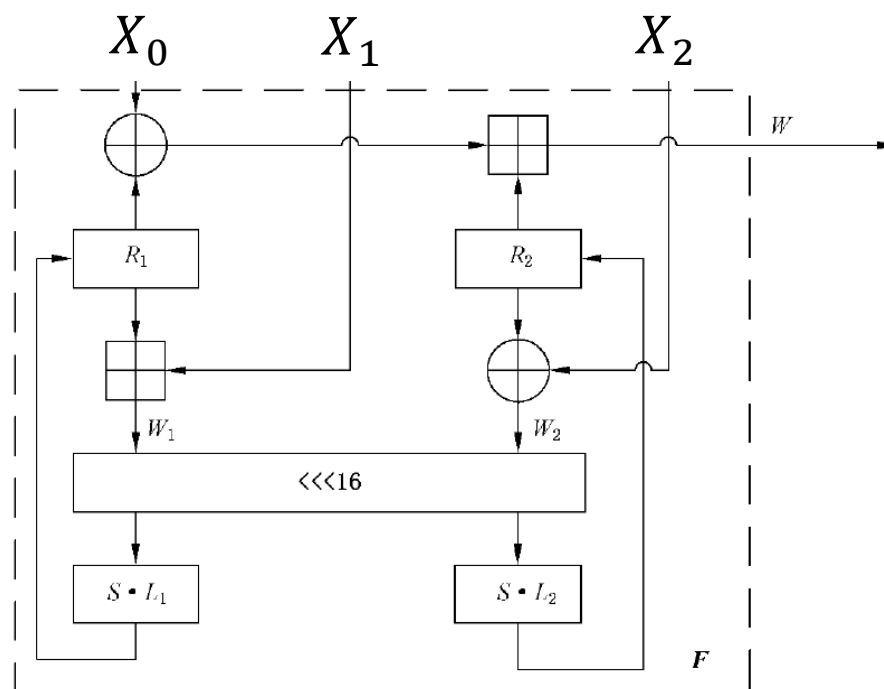
}

H表示取高16位bit

L表示取低16位bit



□ 输入为三个32比特字 X_0, X_1, X_2 ，输出为一个32比特字 W





□ 具体如下所示，其中S表示Sbox变换， L_1, L_2 表示32比特线性变化

$F(X_0, X_1, X_2)$ 表示模 2^{32} 加法运算

{

$$(1) W = (X_0 \oplus R_1) \boxplus R_2;$$

$$(2) W_1 = R_1 \boxplus X_1;$$

$$(3) W_2 = R_2 \oplus X_2;$$

$$(4) R_1 = S[L_1(W_{1L} \parallel W_{2H})];$$

$$(5) R_2 = S[L_2(W_{2L} \parallel W_{1H})]。$$

}

$$L_1(X) = X \oplus (X \ll 2) \oplus (X \ll 10) \oplus (X \ll 18) \oplus (X \ll 24)$$

$$L_2(X) = X \oplus (X \ll 8) \oplus (X \ll 14) \oplus (X \ll 22) \oplus (X \ll 30)$$



□在初始化阶段，算法将 128 比特的 Key 和 IV 载入并处理，然后装入到 LFSR 中
与此同时，设置 32 比特的记忆单元 R_1 和 R_2 （初始化为 0）。然后执行下面的
操作 31 次。

1. $Bitreorganization()$
2. $w = F(X_0, X_1, X_2)$
3. $LFSRWithInitializationMode(w \gg 1)$



□ 初始化完成后，算法立刻进入工作阶段。在该阶段，算法执行下面操作一次，并且丢掉 F 的输出 W：

1. *Bitreorganization()*
2. $F(X_0, X_1, X_2)$
3. *LFSRWithWorkMode()*



□ 然后算法再进入密钥流产生阶段，在每次迭代时，下面操作执行一次，并输出一 次 32 比特的字 Z 作为输出。

1. $Bitreorganization()$
2. $Z = F(X_0, X_1, X_2) \oplus X_3$
3. $LFSRWithWorkMode()$



- 在LFSR设计上ZUC 算法的 LFSR 设计首次采用素域 $GF(2^{31} - 1)$ 的 m 序列。其它常见流密码体制的LFSR均采用二元域或二元域的某个扩域上的 m 序列。这种序列具有明显的多重线性关系，这使得以其为序列源的密码算法容易受到相关攻击。而素域 $GF(2^{31} - 1)$ 中的 m 序列周期长、统计特性好，且在特征为2的有限域上是具有天然的强抵抗二元域上密码攻击方法的能力，譬如二元域上的代数攻击、区分分析和相关攻击等。
- 此外，素域 $GF(2^{31} - 1)$ 上的乘法可以快速实现，LFSR 在设计时充分考虑到安全和效率两方面的问题，在达到高安全目标的同时可以拥有非常高效地软硬件实现。



- 比特重组采用取半合并技术，实现LFSR数据单元到非线性函数F和密钥输出的数据转换，其主要目的是破坏LFSR在素域 $GF(2^{31}-1)$ 上的线性结构。ZUC算法中间层为比特重组。结合下层的非线性函数F，比特重组可使得一些在素域 $GF(2^{31}-1)$ 上的密码攻击方法变得非常困难。
- 由于LFSR和BR都是线性变换，所以非线性函数F就成为ZUC算法中唯一的非线性部件，从而成为确保ZUC算法安全的关键。在F函数设计上在非线性函数F的设计上，ZUC算法设计借鉴了分组密码的设计技巧，提供混淆的S盒和高扩散特性的线性变换L使F函数具有高的抵抗区分分析、快速相关攻击和猜测确定攻击等方法的能力。此外，非线性函数F的S盒采用结构化设计方法，降低了硬件实现代价，具有实现面积小、功耗低等特点。

4.4

国密算法库介绍



库	语言	国密能力	适合的场景
GmSSL	C + Java/Python/Go/PHP	SM2/SM3/SM4/SM9/ZUC, 命令行工具	快速原型、国密功能覆盖
Tongsuo	C	SM2/SM3/SM4/ZUC, TLCP, TLS1.3+国密	网关、HTTPS、VPN、生产协议栈
OpenSSL	C/C++	SM2/SM3/SM4 的 EVP/API 支持	已有 OpenSSL 项目做局部国密改造
Bouncy Castle	Java / C#	SM2Signer/SM2Engine/SM4Engine	Java 业务系统、JCE Provider 集成
Botan	C++	SM2/SM3/SM4, x86/ARMv8 加速	C++ 服务、跨平台工程、性能优化

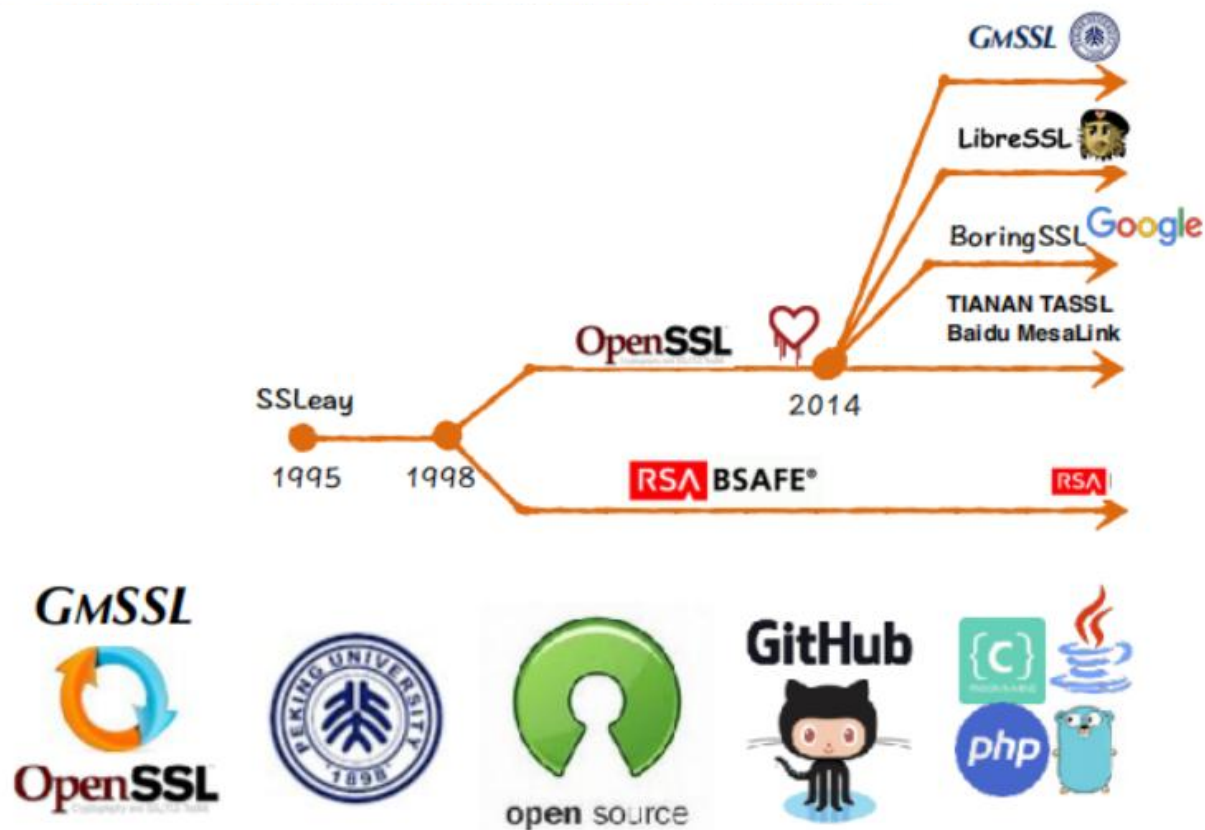


国密算法库介绍

4.4.2 GMSSL项目介绍

B U P T

□ GMSSL是支持国产密码算法的OpenSSL分支。





- 2010年年国家密码管理局公开了包括SM2公钥密码、SM3哈希算法和SM4分组密码等商用密码算法的相关标准(简称国密算法)



- 开发者使用互联网上的零散国密算法代码片段或自行开发，为软件的安全性和可靠性带来了隐患。
- GmSSL是一个提供了完整国密密码的密码库，为开源软件开发者提供了稳定的开发接口和成熟的算法实现，可以促进国产密码算法在国产开源软件领域的进一步应用。



- 开源协议：BSD-like
- 源代码：Github托管 <https://github.com/guanzhi/GmSSL>
- 中文文档：gmssl.org
- 标准文本： <https://github.com/guanzhi/GM-Standards>
- 持续集成(CI)：Mac Linux Windows

GmSSL

 CMake   CMake-windows   CMake-Android   CMake-iOS 

GmSSL是由北京大学自主开发的国产商用密码开源库，实现了对国密算法、标准和安全通信协议的全面功能覆盖，支持包括移动端在内的主流操作系统和处理器，支持密码钥匙、密码卡等典型国产密码硬件，提供功能丰富的命令行工具及多种编译语言编程接口。



- 算法：SM2/SM3/SM4/SM9/ZUC ...
- 接口：SKF/SDF/SAF/SOF...
- 协议：SSL VPN、IPSec VPN
- 规范：技术规范、检测规范

- 主要应用：
 - 中国金融集成电路(IC)卡规范(PBOC3.0)、
 - 公共安全视频监控联网信息安全技术要求、
 - 区块链(联盟链)密码技术标准

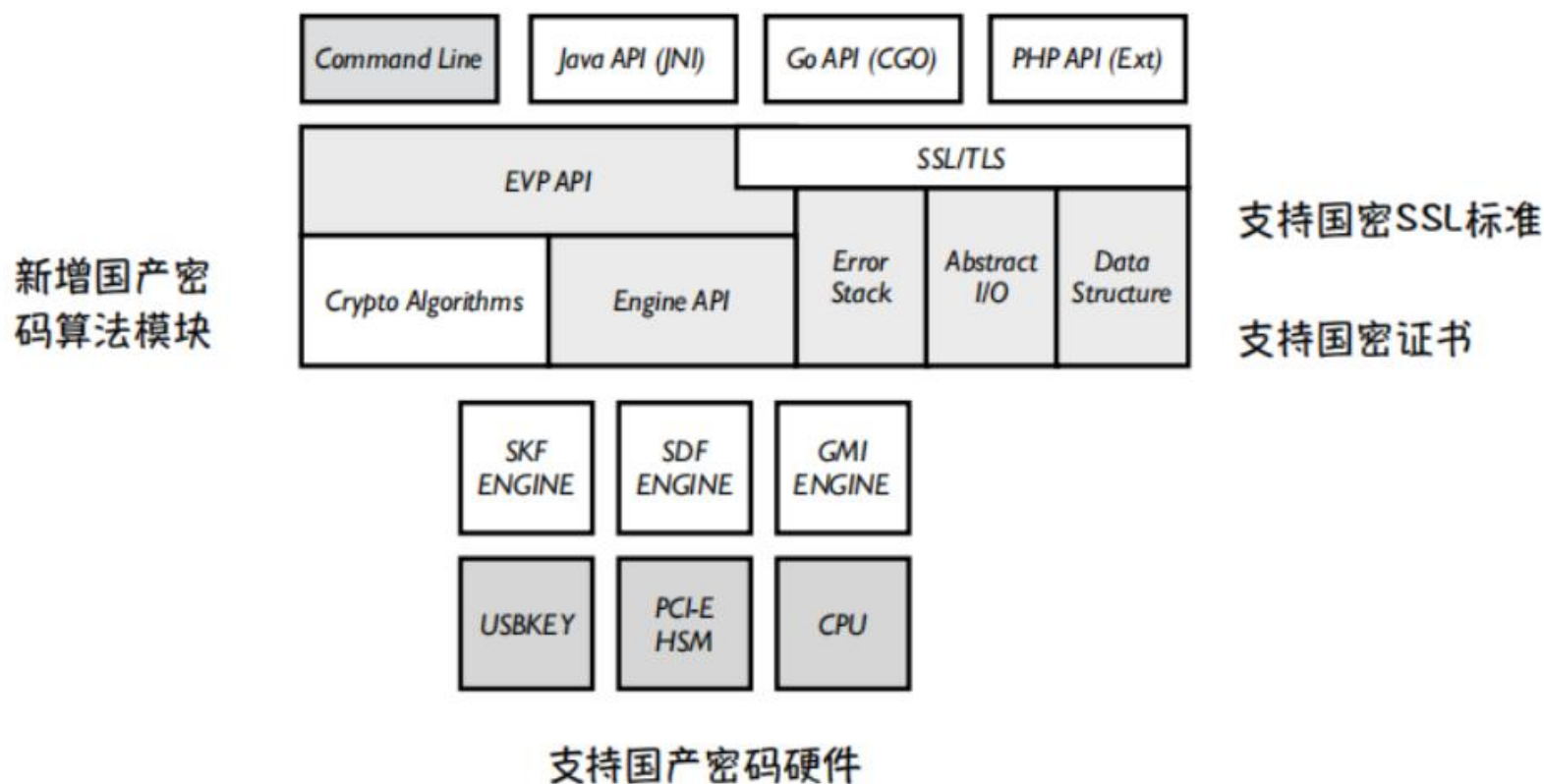


国密算法库介绍

4.4.2 GMSSL系统结构

B U P T

GMSSL系统结构如下所示：





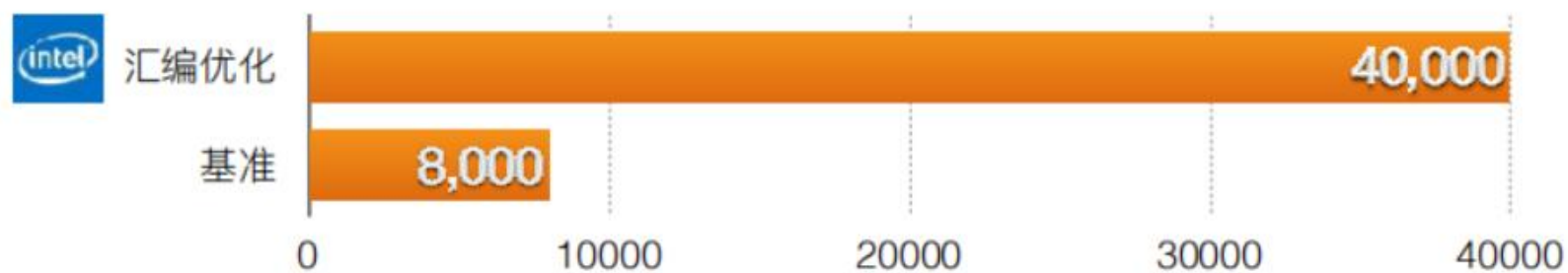
- 全部国密算法 SM1*/2/3/4/9 ZUC SSF33 OTP ...
- SM4支持NIST 13种模式中除FF1和TKW之外的11种
- 国密算法支持抽象的EVP接口
- 国密算法支持X.509/PKCS#5/PKCS#7/PKCS#8
- 覆盖GM/T文本中的测试向量
- 快速实现：SM2/4接近4倍的性能提升
- 不依赖第三方库



国密算法库介绍 4.4.2 GMSSL性能优化

B U P T

□ SM2签名：SM2曲线参数汇编实现+签名算法优化(单位：TPS)



□ SM4加密：汇编实现(单位：MB/S)





国密算法库介绍 4.4.2 GMSSL应用实例

B U P T

- 首先需要安装OpenSSL
- 安装GMSSL: <http://gmssl.org/docs/quickstart.html>
- 下载压缩包并解压, 使用make && makeinstall安装。
- 安装完毕后, 输入gmssl version可查看当前版本

```
lsy@ubuntu: ~  
lsy@ubuntu:~$ gmssl version  
GmSSL 2.5.4 - OpenSSL 1.1.0d 19 Jun 2019  
lsy@ubuntu:~$
```

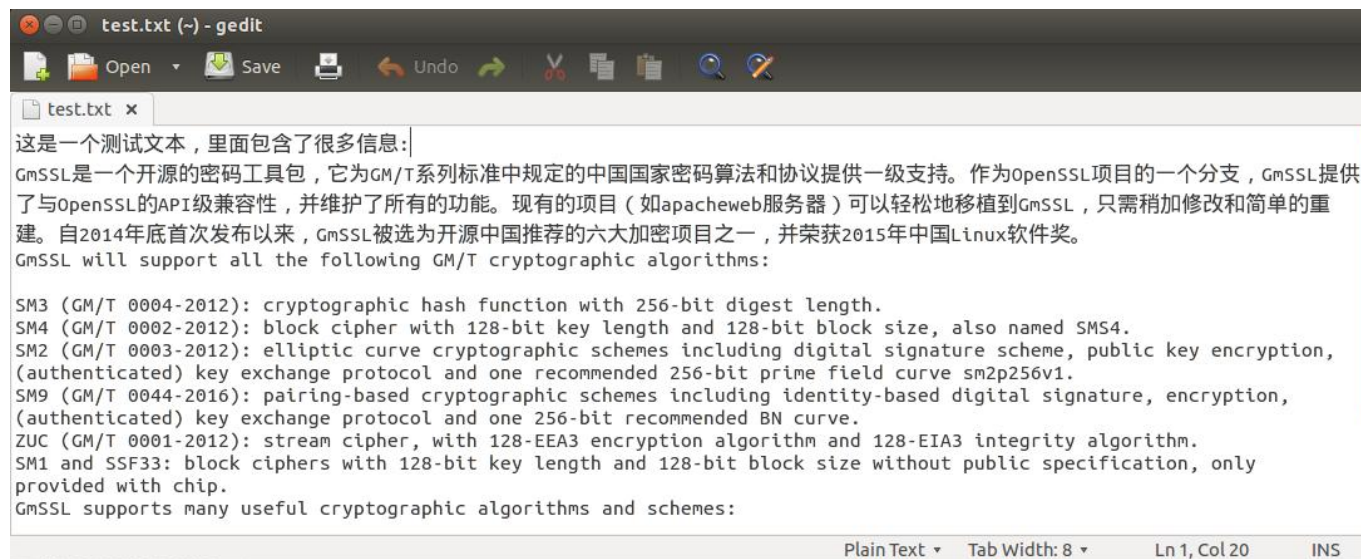


国密算法库介绍

4.4.2 GMSSL应用实例

B U P T

□ 首先创建一个测试文本test.txt



```
test.txt (~) - gedit
Open Save Undo
test.txt x
这是一个测试文本，里面包含了很多信息：
GMSSL是一个开源的密码工具包，它为GM/T系列标准中规定的中国国家密码算法和协议提供一级支持。作为OpenSSL项目的一个分支，GMSSL提供了与OpenSSL的API级兼容性，并维护了所有的功能。现有的项目（如apacheweb服务器）可以轻松地将移植到GMSSL，只需稍加修改和简单的重建。自2014年底首次发布以来，GMSSL被选为开源中国推荐的六大加密项目之一，并荣获2015年中国Linux软件奖。
GMSSL will support all the following GM/T cryptographic algorithms:

SM3 (GM/T 0004-2012): cryptographic hash function with 256-bit digest length.
SM4 (GM/T 0002-2012): block cipher with 128-bit key length and 128-bit block size, also named SMS4.
SM2 (GM/T 0003-2012): elliptic curve cryptographic schemes including digital signature scheme, public key encryption, (authenticated) key exchange protocol and one recommended 256-bit prime field curve sm2p256v1.
SM9 (GM/T 0044-2016): pairing-based cryptographic schemes including identity-based digital signature, encryption, (authenticated) key exchange protocol and one 256-bit recommended BN curve.
ZUC (GM/T 0001-2012): stream cipher, with 128-EEA3 encryption algorithm and 128-EIA3 integrity algorithm.
SM1 and SSF33: block ciphers with 128-bit key length and 128-bit block size without public specification, only provided with chip.
GMSSL supports many useful cryptographic algorithms and schemes:
```

□ 使用SM3生成文本摘要

```
lsy@ubuntu:~$ gmssl sm3 test.txt
SM3(test.txt)= 65e67a570adf89ad9cec5ae65edb60284a4e0ce55c7e1626751f0fabe68ec506
```




国密算法库介绍 4.4.2 GMSSL应用实例

B U P T

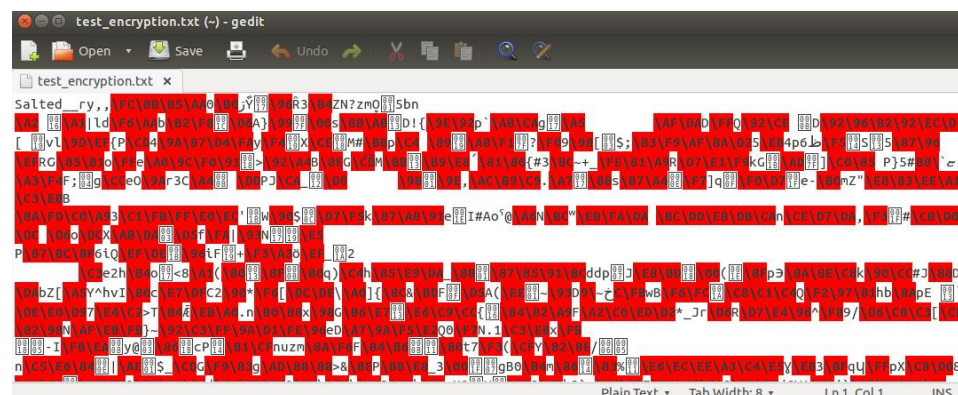
□使用 gmssl sms4 -e -in <inputfile> -out <outputfile>加密文件，并设置加密密码。

```
lsy@ubuntu:~$ gmssl sms4 -e -in test.txt -out test_encryption.txt
enter sms4-cbc encryption password:
Verifying - enter sms4-cbc encryption password:
```

□使用 gmssl sms4 -d -in <inputfile> -out <outputfile>解密文件，并输入加密密码。

```
lsy@ubuntu:~$ gmssl sms4 -d -in test_encryption.txt -out test_decryption.txt
enter sms4-cbc decryption password:
```

□效果如下：





国密算法库介绍 4.4.2 GMSSL应用实例

B U P T

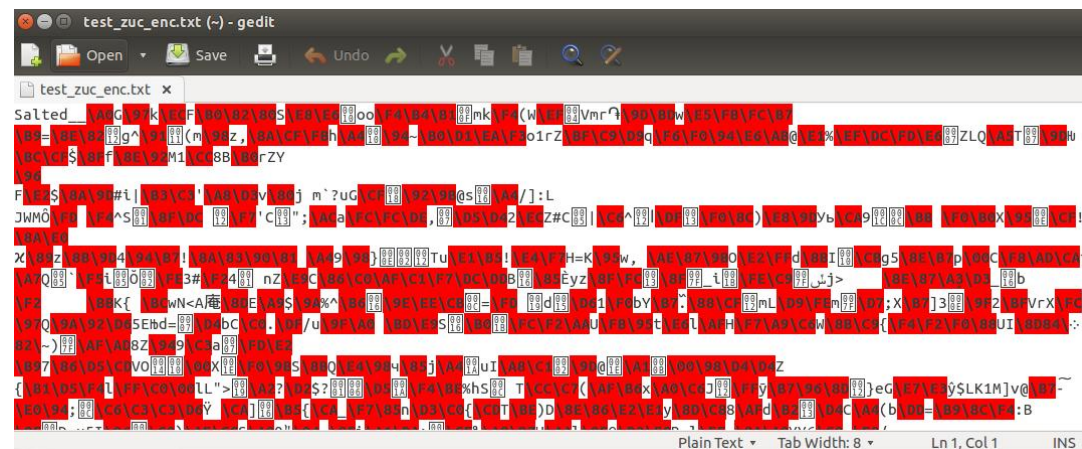
□使用 gmssl zuc -e -in <inputfile> -out <outputfile>加密

```
lsy@ubuntu:~$ gmssl zuc -e -in test.txt -out test_zuc_enc.txt
enter zuc encryption password:
Verifying - enter zuc encryption password:
```

□使用gmssl zuc -d -in <inputfile> -out <outputfile>解密

```
lsy@ubuntu:~$ gmssl zuc -d -in test_zuc_enc.txt -out test_zuc_dec.txt
enter zuc decryption password:
```

□效果如下:





国密算法库介绍 4.4.3 Tongsuo介绍

B U P T

- **铜锁Tongsuo** 是一个提供现代密码学算法和安全通信协议的开源基础密码库，为存储、网络、密钥管理、隐私计算等诸多业务场景提供底层的密码学基础能力，实现数据在传输、使用、存储等过程中的私密性、完整性和可认证性，为数据生命周期中的隐私和安全提供保护能力。
- 技术合规能力
 - 符合GM/T 0028 《密码模块安全技术要求》的“软件密码模块安全一级”资质
 - 符合 GM/T 0005-2021 《随机性检测规范》
- 支持零知识证明 (ZKP)
 - Bulletproofs (Range)
 - Bulletproofs (R1CS)
- 官方文档: <https://www.tongsuo.net/docs/>
- 项目地址: <https://github.com/Tongsuo-Project/Tongsuo>

Tong 铜锁
SUO



□ 密码学算法

中国商用密码算法：SM2、SM3、SM4、祖冲之等

国际主流算法：ECDSA、RSA、AES、SHA 等

同态加密算法：EC-ElGamal、Paillier 等

后量子密码学：ML-KEM(Kyber)、ML-DSA(Dilithium)、SLH-DSA(SPHINCS+)等

□ 安全通信协议

支持 GB/T 38636-2020 TLCP 标准，即双证书国密通信协议

支持 RFC 8998，即TLS 1.3+ 国密单证书

支持 QUIC API

支持 Delegated Credentials 功能，基于 RFC9345

支持 TLS 证书压缩，基于RFC8879



国密算法库介绍 4.4.4 OpenSSL介绍

B U P T

■ **OpenSSL**是一个开放源代码的软件库，提供了丰富的加密算法和工具，如生成密钥、创建证书、加密解密等。它支持多种协议，并且可以轻松地与其他应用程序集成。OpenSSL的名称来源于其创始人“Eric A. Young”的首字母缩写，但目前该项目的开发和维护由一个全球社区负责。OpenSSL采用C语言作为开发语言，支持Linux、Windows、BSD、Mac、VMS等平台。

■ OpenSSL 对国密的支持情况

算法层面：官方 OpenSSL 已支持 SM2、SM3、SM4。其中，SM3 为摘要算法，SM4 支持 CBC、ECB、CTR、CFB、OFB、GCM、CCM、XTS 等模式，SM2 支持签名与加密。

接口层面：OpenSSL 3.x 主要通过 EVP + Provider 机制调用国密算法，适合在通用密码应用中统一接入。

协议层面：官方 OpenSSL 重点支持标准 SSL/TLS/DTLS/QUIC。对 TLCP 和 RFC 8998 国密 TLS 套件，上游官方版本并未作为完整能力提供。

■ **项目地址：**<https://github.com/openssl/openssl>

OpenSSL
Cryptography and SSL/TLS Toolkit



□ 漏洞的发现

2014年4月8日，OpenSSL的漏洞曝光。这个漏洞被曝光的黑客命名为“**heartbleed**”，意思是“心脏流血”——代表着最致命的内伤。利用该漏洞，黑客坐在自己家里电脑前，就可以实时获取到约30%https开头网址的用户登录账号密码，包括大批网银、购物网站、电子邮件等。

□ 漏洞的成因

此次漏洞的成因是OpenSSL Heartbleed模块存在一个BUG，当攻击者构造一个特殊的数据包，满足用户心跳包中无法提供足够多的数据会导致memcpy把SSLv3记录之后的数据直接输出，该漏洞导致攻击者可以远程读取存在漏洞版本的OpenSSL服务器内存中长达64K的数据。

□ 漏洞的影响

由于Windows上用OpenSSL的软件很多，且多数使用自己安装目录下的SSLeay32.dll和Libeay32.dll，没有统一的升级管理机制，该漏洞可能会长期存在。全球范围内，据扫描显示约有4000万台服务器数据暴露；其中在中国，受到该漏洞影响的服务器接近三万台。

[漏洞的探测和利用](#)



国密算法库介绍 4.4.5 Bouncy Castle 介绍

B U P T

■ **Bouncy Castle** 是由澳大利亚慈善组织维护的Java平台开源轻量级密码术包，提供JCE 1.2.1规范的实现，支持包括对称加密（AES、DES）、非对称加密（RSA、ECC）、数字签名（ECDSA）、消息摘要（SHA系列）及X.509证书管理等密码学功能。其轻量化设计兼容J2SE 1.4至J2ME（含MIDP）平台，是MIDP环境唯一的完整密码解决方案。

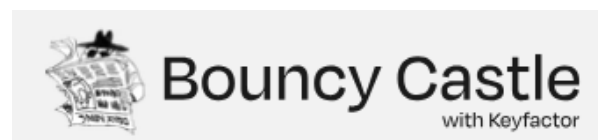
■ Bouncy Castle 支持国密情况

算法支持：支持 SM2、SM3、SM4、ZUC

Java 侧能力：支持国密算法调用，并新增 SM2 Key Exchange、SM4 Key Wrap/Unwrap

协议支持：BCJSSE 提供 RFC 8998 下的国密 TLS 1.3 实验性支持，但默认未开

■ 官方文档：<https://www.bouncycastle.org/documentation/>





■ Botan是一个功能强大的C++加密工具包，它为开发者提供了完整的加密解决方案。无论你是需要实现TLS协议、处理X.509证书，还是进行现代加密算法开发，Botan都能满足你的需求。这个开源库采用简化的BSD许可证，支持多种加密标准，包括后量子密码学、PKCS#11硬件接口和TPM设备访问等高级功能。

■ Botan 支持国密情况

支持 SM2、SM3、SM4 等核心国密算法

提供 C++ / C / Python API，适合应用层国密算法接入

对 SM3、SM4 提供多种硬件加速优化实现

自带完整 TLS 1.2 / 1.3 / DTLS 1.2 栈

■ 官方文档：<https://github.com/randombit/botan>

Botan



3.GMSSL有什么主要的特性?

谢谢大家
